

# Problem Set 5

## A. Jumping on Walls

tianzheyu submissions

| tianzheyu submissions     |                          |           |                                      |                          |          |        |         |
|---------------------------|--------------------------|-----------|--------------------------------------|--------------------------|----------|--------|---------|
| #                         | When                     | Who       | Problem                              | Lang                     | Verdict  | Time   | Memory  |
| <a href="#">382867103</a> | Jul/16/2026 11:31 UTC+10 | tianzheyu | <a href="#">B - Jumping on Walls</a> | C++23 (GCC 14-64, msys2) | Accepted | 186 ms | 8100 KB |

<https://codeforces.com/submissions/tianzheyu#>

### Process

I implemented a Breadth-First Search (BFS) to find the shortest path out of the canyon.

### Challenges and Overcoming

When coding my BFS, I ran into a bug where the water level was rising completely incorrectly. I originally used a global height variable that incremented every time a single node was popped from the queue. I found it when I realized that nodes on the exact same depth level of the BFS were experiencing different water heights. Next time to prevent this bug, I'll strictly bind the state together by storing it as a pair `(current_node, depth_of_bfs(the_time))` inside the queue, ensuring time only increments per step taken.

I also ran into an edge-case bug where my program would never let the ninja escape. To check if a node was underwater, I used `next_node % n`. However, the winning node `2 * n` evaluated to height `0`, so the code always thought the escape route was flooded. Next time to prevent this bug, I'll make sure to explicitly separate and handle win-condition edge cases before applying general bounds and validity checks.